A Trimester – 1 Class Project Report on

INDIAN RAILWAY RESERVATION SYSTEM

Submitted in the partial fulfillment of the requirements for

26CS5101E – Computational Thinking and Programming

Master of

Technology In

Artificial Intelligence & Data Science

By

Student ID: 2601150026

Student Name : YADLA SYAM SUNDAR

under the supervision of

Dr. G. Syam Prasad

Professor



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd., u/s 3 of UGC Act 1956)

Greenfields, Vaddeswaram, Guntur (Dist.), Andhra Pradesh - 522502

2026 – 27

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled "**INDIAN RAILWAY RESERVATION SYSTEM**" is being submitted by **YADLA SYAM SUNDAR** and **Reg. no: 2601150026** submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University is a record of Bonafide work carried out under our guidance and supervision during the academic year 2026 – 2027.

Dr. G. Syam Prasad

Project Supervisor

Signature

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DECLARATION

The Trimester -1 Class Project Report entitled "**INDIAN RAILWAY RESERVATION SYSTEM**" is a record of Bonafide work of **YADLA SYAM SUNDAR** and **Reg.no: 2601150026** submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University.

YADLA SYAM SUNDAR - 2601150026

ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the trimester - 1 class project report.

I express sincere gratitude to HOD of AI&DS Dept. **Dr. M. Venkata Naresh** for his leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. G. Syam Prasad** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

ABSTRACT

The Indian Railway Reservation System (IRRS) is a Python console-based academic prototype that demonstrates the core workflow of a railway ticket reservation application. Users can search for sample trains by source and destination, inspect fares, reserve available seats for a chosen date, receive a unique Passenger Name Record (PNR), check booking status and cancel confirmed tickets.

The system represents trains and bookings with typed data classes, indexes trains and bookings in dictionaries, uses sorting for route results, persists booking records to JSON and protects the critical booking operation with a re-entrant lock. Unit tests validate search, booking, cancellation and concurrent seat allocation. The design deliberately excludes live train data, payment processing, identity verification and external railway integrations.

The project is organized into six assessment components: problem solving and computational thinking; programming and data structures; software design and engineering; functional programming and concurrency; software quality and DevOps practices; and AI-assisted software engineering.

Keywords

Python, railway reservation, PNR, data structures, object-oriented design, threading, JSON, pytest, computational thinking.

INDEX / TABLE OF CONTENTS

Section	Topic	Page
Front matter	Title, certificate, declaration, acknowledgement, abstract	1–5
1	Index and project information	6–7
2	Introduction, objectives and requirements	8–9
3	System design and workflow	10–11
Component 1	Problem Solving & Computational Thinking	12–13
Component 2	Programming & Data Structures	14–15
Component 3	Software Design & Engineering	16–17
Component 4	Functional Programming & Concurrency	18–19
Component 5	Software Quality & DevOps Practices	20–21
Component 6	AI-Assisted Software Engineering	22
4	Execution, outputs and technical defence	23
5	Conclusion, future scope and references	24

PROJECT INFORMATION

Field	Description
Project title	Indian Railway Reservation System (IRRS)
Category	Academic Python software project
Application type	Menu-driven command-line application
Language	Python 3.10+
Data storage	Local JSON file
Testing	pytest
Modules	railway.py, test_railway.py
Users	Student / evaluator using the console
Status	Working demonstration prototype

Scope

The implementation models a limited sample railway network. Route search matches origin and destination exactly after case normalization. Seat counts are maintained separately for each train and journey date. A booking has a generated PNR and one of two statuses: CONFIRMED or CANCELLED.

Out of scope

Live train schedules, actual seat maps, payment gateway, RAC/waitlist, real passenger authentication, refunds, railway APIs and production database infrastructure.

INTRODUCTION

Railway ticket reservation is a familiar example of a transaction-processing system. A passenger needs to identify a suitable train, verify whether seats are available on a particular date, create a booking and retrieve the reservation later. Multiple passengers may try to reserve the final seat at the same time, making consistency an important engineering concern.

Problem statement

Design and implement a maintainable Python application that handles the essential steps of searching, reserving, enquiring and cancelling while preventing seat over-allocation under concurrent requests.

Motivation

This domain makes algorithmic choices visible: dictionaries support fast lookup, filtering and sorting support search, immutable train records prevent accidental edits, and locking protects a shared resource. The implementation is compact enough to explain and demonstrate during an oral defence.

OBJECTIVES AND REQUIREMENTS

Objectives

Decompose a real-world booking workflow into independent functions.

Select suitable Python data structures and algorithms.

Apply classes, data classes, type hints and composition.

Introduce thread-safe seat reservation.

Persist booking information and test critical behaviours.

Review AI-generated suggestions against explicit acceptance criteria.

Functional requirements

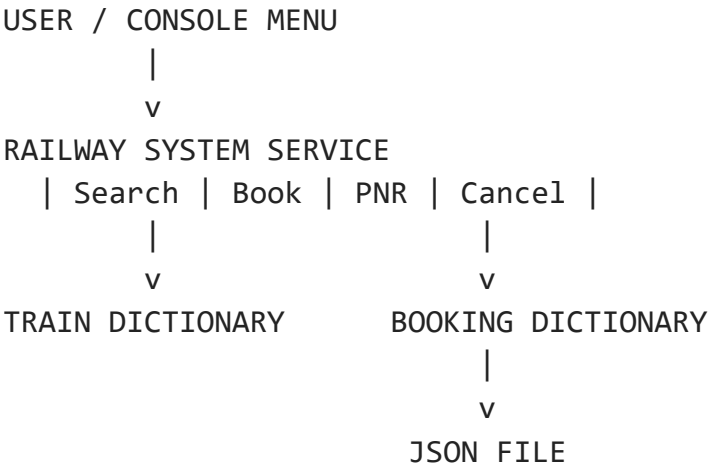
ID	Requirement
FR1	Search sample trains by origin and destination
FR2	Show train number, name and fare
FR3	Book a seat for a train and date
FR4	Generate a unique PNR
FR5	Look up booking by PNR
FR6	Cancel a confirmed booking

Non-functional requirements

Correctness, readable code, modularity, predictable input validation, simple persistence, concurrency safety within a single application process and testability.

SYSTEM ARCHITECTURE

The application uses a layered, single-process architecture. The console menu is responsible for user interaction; the RailwaySystem service contains business rules; typed Train and Booking models hold state; and a JSON file stores bookings between runs.



Data flow

Input is normalized or validated before business rules run. The service looks up train information, computes availability for the selected date, and updates booking records under a lock. The JSON file is replaced after successful changes.

Design assumption

This lock coordinates threads sharing one RailwaySystem instance. It does not provide cross-process database transactions.

WORKFLOW AND ALGORITHMS

Reservation workflow

```
 START
Read train number, passenger, age, date
Validate input and find train
Acquire booking lock
Count confirmed bookings for train and date
IF count >= capacity: reject request
ELSE: generate PNR, create booking, save JSON
Release lock
Display PNR
 END
```

Cancellation workflow

```
Read PNR -> find booking -> acquire lock
IF confirmed: change status to CANCELLED
Save file -> release lock -> show result
```

Sample data

Train	Route	Capacity	Demo fare (INR)
12727	Visakhapatnam–Hyderabad	3	850
12760	Hyderabad–Chennai	4	780
12839	Chennai–Howrah	2	1100
12622	New Delhi–Chennai	3	1450

Sample trains, prices and capacities are invented for demonstration and must not be interpreted as live railway data.

COMPONENT 1: PROBLEM SOLVING & COMPUTATIONAL THINKING

Decomposition and abstraction

The real-world booking problem is divided into route discovery, capacity checking, passenger validation, reservation creation, PNR enquiry and cancellation. A Train abstracts schedule identity, route, capacity and fare; a Booking abstracts passenger details and reservation status.

Algorithm selection

A linear scan filters the small train catalogue; sorted results give a deterministic fare-first order. A dictionary maps PNR to bookings. Availability is calculated by counting only confirmed bookings matching both train and date. A lock makes the check-and-update sequence atomic with respect to threads using the same service instance.

Complexity

Operation	Time	Space
Train lookup by number	Expected O(1)	O(T) catalogue
Route search	O(T + K log K)	O(K) results
PNR lookup	Expected O(1)	O(B) bookings
Availability	O(B)	O(1) extra
Book / cancel	O(B) plus JSON save O(B)	O(B) serialization

T = number of trains, B = number of bookings, K = matching trains. Worst-case hash-table operations may degrade; expected values assume normal hashing.

COMPONENT 1: ALGORITHM CODE & ANALYSIS

```
def search(self, source: str, destination: str) -> list[Train]:
    return sorted(
     (t for t in self.trains.values()
         if t.source == source.strip().upper()
      and t.destination == destination.strip().upper()),
     key=lambda t: (t.fare, t.number)
 )

def available(self, number: str, date: str) -> int:
 train = self.trains[number]
 used = sum(
     b.status == 'CONFIRMED'
     and b.train_number == number
     and b.journey_date == date
        for b in self.bookings.values()
 )
    return train.seats - used
```

Explanation

The generator expression avoids allocating a separate intermediate list during filtering. Sorting ranks the matches by fare and then train number. Availability checks a specific train/date pair, so reservations on other dates do not consume seats. A production system would use indexed database counts or inventory rows rather than scanning all bookings.

Algorithmic-pattern judgement

Greedy sorting is appropriate for presenting cheapest fares first, but the prototype does not solve a multi-leg route optimization problem. Dynamic programming, backtracking and branch-and-bound would add unnecessary complexity for the current direct-route scope.

COMPONENT 2: PROGRAMMING & DATA STRUCTURES

Selected structures

Structure	Purpose	Why chosen
dict[str, Train]	Train number lookup	Expected constant-time access
dict[str, Booking]	PNR lookup and mutation	Simple keyed storage
list[Train]	Sorted route results	Ordered output
dataclass	Structured records	Readable fields and conversion
JSON array	Durable local records	Portable and inspectable

Python techniques

The project uses type hints, dictionary comprehensions during loading, generator expressions for counting and filtering, dataclasses for concise models, context-managed thread pools in tests and exception handling in the command-line interface. A standalone iterator or decorator is not necessary for the application’s requirements.

Persistence note

JSON is adequate for a classroom prototype. For larger deployments, a transactional database and unique constraints would be more reliable.

COMPONENT 2: PYTHON CODE

```
from dataclasses import dataclass, asdict
from pathlib import Path
import json

@dataclass(frozen=True)
class Train:
 number: str
 name: str
 source: str
 destination: str
 seats: int
 fare: int

@dataclass
class Booking:
 pnr: str
 train_number: str
 passenger: str
 age: int
 journey_date: str
 status: str = 'CONFIRMED'

def _save(self) -> None:
 temporary = self.path.with_suffix('.tmp')
 temporary.write_text(
     json.dumps([asdict(b) for b in self.bookings.values()],
                indent=2), encoding='utf-8'
 )
 temporary.replace(self.path)
```

Explanation

A frozen Train record keeps catalogue data immutable. Booking remains mutable because its status changes on cancellation. The asdict conversion produces JSON-serializable dictionaries. Writing a temporary file and replacing the target reduces the risk of leaving a partially written file after interruption, but it does not provide multi-process isolation or a full crash-consistency guarantee.

COMPONENT 3: SOFTWARE DESIGN & ENGINEERING

Object-oriented model

RailwaySystem composes train and booking dictionaries. Train and Booking are domain models, while RailwaySystem owns validation, business rules, synchronization and persistence. The menu function acts as a thin presentation layer. This separation reduces duplication and allows tests to call the service directly.

Design principles

Encapsulation: methods expose search, booking, cancellation and enquiry.

Composition: RailwaySystem contains Train and Booking instances.

Type hints: interfaces describe input and output types.

Single responsibility: each method performs one primary task.

Dependency injection: the persistence path can be passed to the constructor.

Patterns and limitations

The service resembles a simple façade over booking operations. The model does not introduce inheritance because no meaningful subclass relationship is required. Adding abstract base classes or generic repositories would be possible in a larger system, but unnecessary abstraction would make this teaching prototype harder to understand.

COMPONENT 3: OBJECT-ORIENTED CODE

```
class RailwaySystem:
    def __init__(self, path: str = 'bookings.json') -> None:
     self.path = Path(path)
     self.lock = RLock()
     self.trains: dict[str, Train] = {
         t.number: t for t in [
             Train('12727', 'Godavari Express',
                   'VISAKHAPATNAM', 'HYDERABAD', 3, 850),
             Train('12760', 'Charminar Express',
                   'HYDERABAD', 'CHENNAI', 4, 780),
             Train('12839', 'Chennai Mail',
                   'CHENNAI', 'HOWRAH', 2, 1100),
             Train('12622', 'Tamil Nadu Express',
                   'NEW DELHI', 'CHENNAI', 3, 1450),
         ]
     }
     self.bookings: dict[str, Booking] = {}
     self._load()

    def get_booking(self, pnr: str) -> Booking | None:
        return self.bookings.get(pnr.upper())
```

Explanation

The constructor assembles the application state and loads saved bookings. The service owns all operations on its collections, so callers need not implement reservation rules. The type Booking | None communicates that an unknown PNR is a normal outcome.

COMPONENT 4: FUNCTIONAL PROGRAMMING & CONCURRENCY

Functional techniques

Filtering, generator expressions, sum and sorted are functional-style operations. A sort-key lambda is a pure function of a Train record. The code minimizes side effects in query operations; state changes are deliberately concentrated in book and cancel.

Race condition

Without synchronization, two threads might both observe one available seat and create two bookings. The reservation operation therefore places availability checking, PNR creation, insertion and persistence inside one re-entrant lock.

Why threading?

Threading is appropriate here to demonstrate coordination of concurrent requests and I/O-bound activity. The global interpreter lock (GIL) does not make multi-step booking operations atomic. Multiprocessing is not used because CPU-heavy parallel calculation is not required. Async programming is possible but adds little benefit to this synchronous console application.

COMPONENT 4: CONCURRENT BOOKING CODE

```
def book(self, number: str, name: str,
       age: int, date: str) -> Booking:
  datetime.strptime(date, '%Y-%m-%d')
     if number not in self.trains:
      raise ValueError('Unknown train number')
     if not name.strip() or not 1 <= age <= 120:
      raise ValueError('Invalid passenger name or age')
     with self.lock:
         if self.available(number, date) <= 0:
          raise ValueError('No seats available')
      pnr = uuid.uuid4().hex[:12].upper()
      booking = Booking(pnr, number, name.strip(), age, date)
      self.bookings[pnr] = booking
      self._save()
         return booking

def cancel(self, pnr: str) -> bool:
     with self.lock:
      booking = self.bookings.get(pnr.upper())
         if not booking or booking.status != 'CONFIRMED':
             return False
      booking.status = 'CANCELLED'
      self._save()
         return True
```

Explanation

The with statement acquires and releases the lock, even if an exception occurs. Validation rejects unknown trains, blank names and invalid ages. UUID-derived PNRs are highly unlikely to collide in this small demo; a production system must enforce uniqueness explicitly.

COMPONENT 5: SOFTWARE QUALITY & DEVOPS PRACTICES

Test strategy

Test level	Example	Expected result
Unit	Search by route	Matching sample train returned
Unit	Reserve one seat	Availability decreases by one
Unit	Cancel booking	Availability increases by one
Integration	Book then read PNR	Persisted booking is retrievable
Concurrency	Ten requests for two seats	Exactly two confirmed
Negative	Invalid age / train	Error shown; no booking

Quality tools

pytest executes automated tests. Ruff can lint and format source files; mypy can perform static type checking; pytest-cov can report coverage. These optional tools are suggested engineering practices, not claims that they were executed for this report.

Commands

```
python railway.py
python -m pytest -q
python -m pip install ruff mypy pytest-cov
ruff check railway.py test_railway.py
ruff format --check railway.py test_railway.py
mypy railway.py
python -m pytest --cov=railway --cov-report=term-missing
```

COMPONENT 5: TEST CODE & CI/CD

```
def test_booking_and_cancel(tmp_path):
 system = RailwaySystem(str(tmp_path / 'bookings.json'))
 booking = system.book('12727', 'Anu', 21, '2026-12-01')
    assert system.available('12727', '2026-12-01') == 2
    assert system.get_booking(booking.pnr) is not None
    assert system.cancel(booking.pnr)
    assert system.available('12727', '2026-12-01') == 3

def test_no_overbooking_under_threads(tmp_path):
 system = RailwaySystem(str(tmp_path / 'bookings.json'))
    def attempt(i):
        try:
            return system.book('12839', f'User{i}', 25,
                               '2026-12-01')
        except ValueError:
            return None
    with ThreadPoolExecutor(max_workers=10) as pool:
     results = list(pool.map(attempt, range(10)))
    assert sum(x is not None for x in results) == 2
```

Execution result

Local test execution during report preparation: 3 passed. Tests cover route search, book-and-cancel and concurrent overbooking protection. This is not a claim of full branch coverage or production readiness.

CI/CD and containerization

A CI pipeline could install Python, install pytest, run tests, then run lint and type checks on each commit. Containerization is optional for this small console program. A container image would need a mounted data volume to preserve bookings between runs.

COMPONENT 6: AI-ASSISTED SOFTWARE ENGINEERING

Specification-first development

Before using AI assistance, define exact operations, expected input/output, invalid cases, concurrency rules and testing criteria. AI suggestions are drafts rather than evidence of correctness. The student must read, run, test and explain every submitted component.

Example prompt

“Create a Python 3.10+ educational railway reservation console app with typed Train and Booking data classes, JSON persistence, search, date-specific availability, PNR enquiry, cancellation, a thread-safe booking critical section and pytest tests. Do not use live railway APIs or real payment data. Explain assumptions and edge cases.”

Review checklist

Verify that the date-specific seat count never becomes negative.

Check that all shared-state changes occur under the lock.

Test cancellation of unknown and already cancelled PNRs.

Confirm that sample train records are clearly fictional.

Inspect generated code for unnecessary dependencies and security issues.

Retest after every AI-assisted edit and document unresolved limitations.

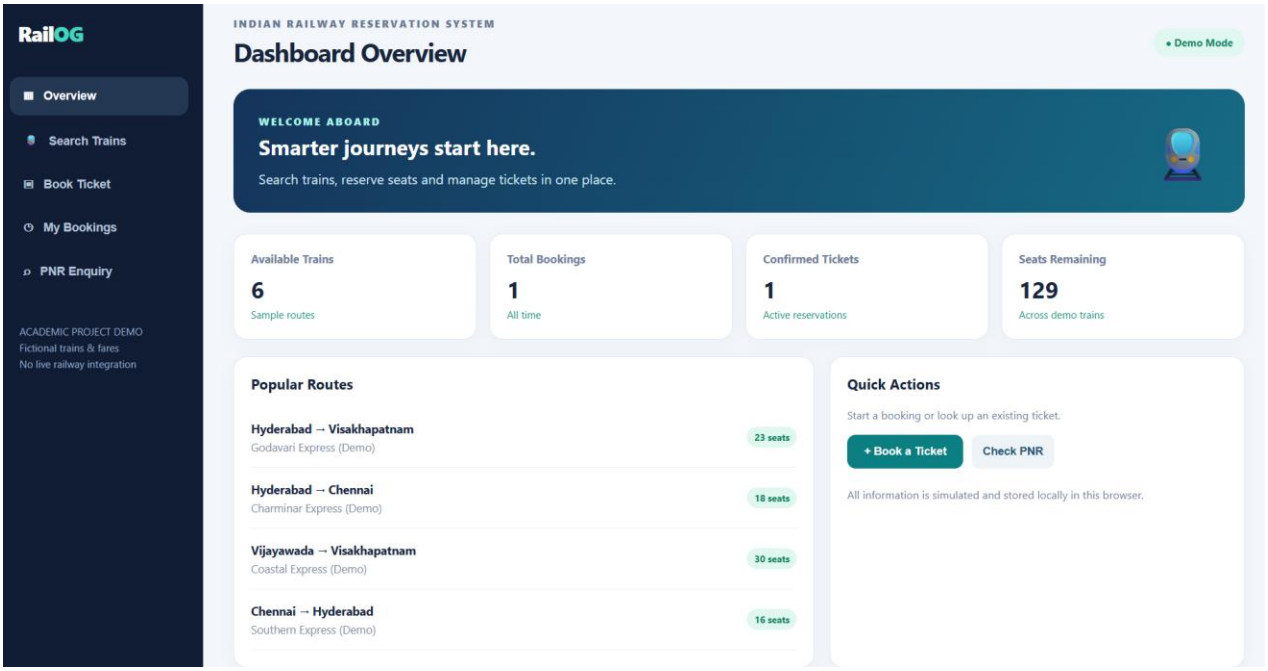
How to run

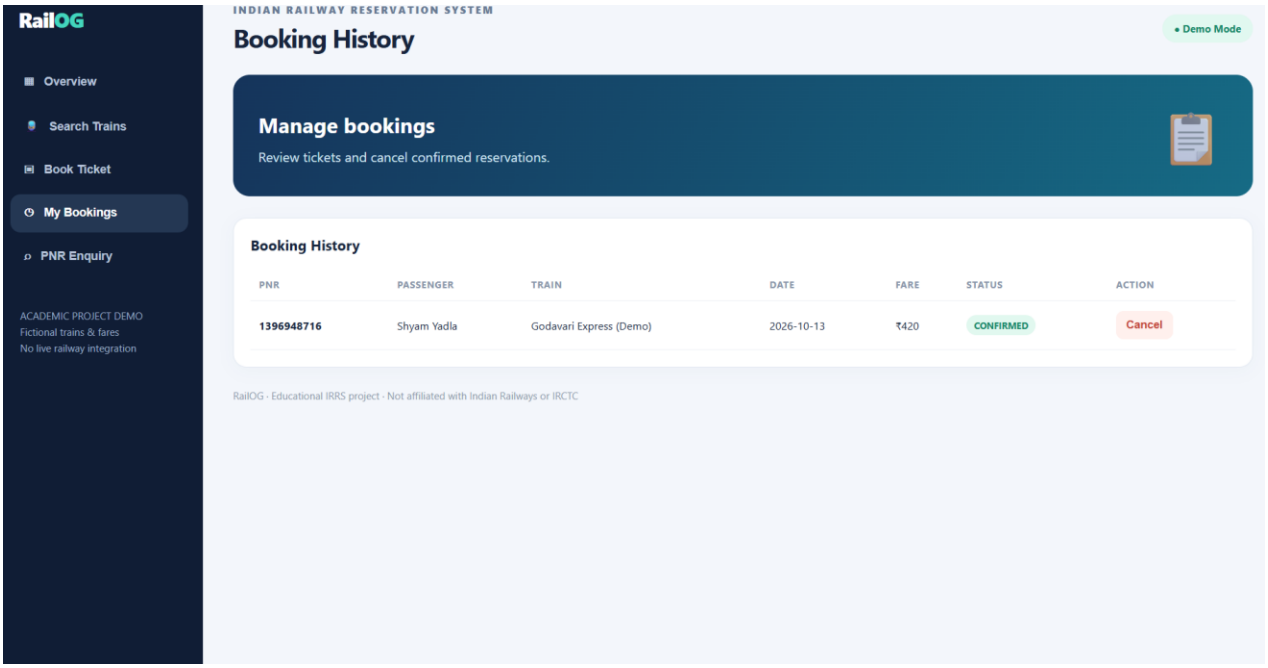
```
cd IRRS_Project
python railway.py
# Menu: 1 Search  2 Book  3 PNR  4 Cancel  5 Exit
```

Illustrative console session

```
INDIAN RAILWAY RESERVATION SYSTEM (DEMO)
   Select option: 1
   From: Visakhapatnam
   To: Hyderabad
   Train(number='12727', name='Godavari Express', ...)
   Select option: 2
Train number: 12727
Passenger name: Anu
Age: 21
Journey date (YYYY-MM-DD): 2026-12-01
   Confirmed. PNR: <generated 12-character identifier>
   Select option: 3
   PNR: <the generated identifier>
   Booking(... status='CONFIRMED')
```

Dashboard and Ticket booking:





CONCLUSION, FUTURE SCOPE & REFERENCES

Conclusion

The IRRS prototype demonstrates how a familiar real-world reservation problem can be decomposed into a small, maintainable Python solution. It combines search algorithms, dictionary-based indexing, object-oriented models, functional-style queries, thread synchronization, file persistence and automated testing. The implementation is suitable for explaining the six course-assessment components during a live project defence.

Future scope

Replace JSON with PostgreSQL or another transactional database.

Introduce verified passenger login, access control and audit trails.

Add RAC/waitlist, quotas, class selection and multi-passenger PNRs.

Use official authorized railway data feeds if available.

Add a web UI, accessibility checks, structured logs and CI/CD.

Improve persistence failure handling and cross-process concurrency.

References

1. Uploaded “Project Defence & Demonstration — Computational Thinking and Programming” assessment document (2 pages).

2. Python Software Foundation. Python 3 documentation: https://docs.python.org/3/

3. pytest documentation: https://docs.pytest.org/

4. Python documentation for threading, dataclasses, pathlib, json and unittest concepts.

Full executable source and tests are supplied separately in the IRRS_Project folder. Component-wise report listings are selected excerpts from the executable source.